BUILDING AI AGENTS WITH MCP

From Agent Loop to Production Servers — O'Reilly Live Training Cheatsheet

Model Context Protocol · Claude Agent SDK · MCPServer · Deployment · MCP Apps

1. WHAT IS MCP?

H	C	S	T
Host	**Client**	**Server**	**Transport**

MCP Host

Application that wants AI + tool access. Examples: Claude Desktop, VS Code, Cursor. Coordinates multiple MCP client connections.

MCP Client

Protocol layer with 1:1 connection to one server. Handles JSON-RPC communication. Built into the host application.

MCP Server

Lightweight program exposing tools, resources, or prompts. Runs as a separate process (or in-process). Speaks the MCP protocol.

Transport Layer

stdio — subprocess pipe for local servers. **Streamable HTTP** — remote/deployed servers (stateless-capable). *HTTP+SSE is deprecated since 2025-03.*

The Analogy

MCP is the **"USB-C port for AI"** — one universal standard that lets any AI model plug into any tool, database, or service. The "HTTP moment" happened: MCP joined the **Linux Foundation** (Agentic AI Foundation, Dec 2025) with Anthropic, OpenAI, Google, Microsoft and AWS on board. Current spec: **2026-07-28** — the largest revision since launch, now shipped (stateless core, extensions, multi round-trip requests). This course uses SDK v2, which speaks both **2026-07-28** and **2025-11-25**.

2. THE MCP PRIMITIVES (+ THE APPS EXTENSION)

Tools — Model-controlled

Functions the AI decides to call. Like function calling — LLM triggers execution. `@mcp.tool()`

Resources — Application-controlled

Data the host app exposes to context. Files, DB rows, API data. URI-addressed. `@mcp.resource("uri://")`

Prompts — User-controlled

Reusable prompt templates users invoke. Slash commands in Claude Desktop. `@mcp.prompt()`

MCP Apps — UI extension (2026)

Tool + `ui://` resource linked via `_meta` → interactive HTML rendered in-chat (sandboxed iframe). `meta={"ui": {"resourceUri": "ui://…"}}`

Deprecation watch (2026-07-28 spec)

Sampling, Roots, and Logging enter deprecation (12-month sunset) — don't build new work on them. Long-running work moves to the **Tasks** extension.

3. MCP LIFECYCLE

1

Initialize

Client sends capabilities + version. Server responds with its capabilities.

2

Handshake

Client sends `notifications/initialized`. Both sides ready.

3

Operate

Tool calls, resource reads, prompt requests — all via JSON-RPC 2.0. (2026-07-28 made the core stateless: no handshake needed.)

4

Shutdown

Client sends `notifications/closed`. Server cleans up.

4. CORE CODE PATTERNS

MCPServer Pattern

```
#!/usr/bin/env -S uv run --script
# /// script
# requires-python = "⩾3.10"
# dependencies =
["mcp[cli]⩾2.3,<3"]
# ///  # v1 called this class
FastMCP

from mcp.server.mcpserver import
MCPServer

mcp = MCPServer("server-name")

@mcp.tool()
def my_tool(param: str) → str:
    """Description visible to
LLM"""
    return result

@mcp.resource("data://{key}")
def get_resource(key: str) →
str:
    """Resource description"""
    return load_data(key)

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Claude Agent SDK Pattern

```
from claude_agent_sdk import (
    ClaudeAgentOptions, query,
    create_sdk_mcp_server, tool,
)

# In-process tool (no subprocess)
@tool("analyze", "Analyze data.",
{"data": str})
async def analyze(args):
    return {"content": [
        {"type": "text", "text":
run(args["data"])}
    ]}

server = create_sdk_mcp_server(
    name="analysis",
tools=[analyze])

options = ClaudeAgentOptions(
    system_prompt="You are an
analyst.",
    mcp_servers={
        "analysis":
server,                 # in-
process
        "research": {"command":
"uv",       # external stdio
                     "args":
["run", "mcp_server.py"]},
        "prod": {"type":
"http",            # remote
                 "url":
"https://…/mcp"},
    },

allowed_tools=["mcp__analysis__*"
,

"mcp__research__*"],
)
async for msg in
query(prompt="Task",
options=options):
    ...
```

JSON-RPC 2.0 — Tool Call

```
{
  "jsonrpc": "2.0",
  "id": "req-1",
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {"city":
"Lisbon"}
  }
}

// Response
{
  "jsonrpc": "2.0",
  "id": "req-1",
  "result": {
    "content": [
      {"type": "text", "text":
"22°C, Sunny"}
    ]
  }
}
```

Security: PreToolUse Hook

```
 from claude_agent_sdk import
HookMatcher

async def validate(input_data,
tool_use_id, ctx):
    """Deny bad inputs BEFORE the
tool runs."""
    path =
input_data.get("tool_input", {})
\
                      .get("path",
"")
    if ".." in path or
path.startswith("/"):
        return
{"hookSpecificOutput": {
            "hookEventName":
"PreToolUse",
            "permissionDecision":
"deny",

"permissionDecisionReason":
                "Path must be
relative"}}
    return {}

options = ClaudeAgentOptions(
    hooks={"PreToolUse":
[HookMatcher(

matcher="mcp__research__*",
        hooks=[validate])]},
    ...
)
```

5. COURSE MODULE PROGRESSION

7 Modules, One Use Case (a research assistant)

#	Module	What You Build	Key Concept
00	Agents Are Loops	Hand-rolled agent loop	What every framework does inside
01	Intro to MCP	First MCPServer + thin client	stdio, Inspector, M×N payoff (2 hosts)
02	Agent SDK Host	Same server, ~15-line agent	SDK as MCP host; in-process servers
03	Skills vs MCP	Agent scaffolds an MCP server	MCP = access, skills = know-how
04	Production Shape	HTTP server + authed, hooked agent	Intent-grouped tools, auth seam, evals
05	Deploy + MCP Apps	Remote server on Vercel + in-chat UI	Stateless HTTP; one server, many hosts
06	Defend & Scale	Poisoning lab; multi-server team	Tool poisoning; subagents; sessions

6. SECURITY CHECKLIST & COMMON GOTCHAS

Security Checklist

- Validate file paths — prevent directory traversal (`..` , leading `/`)
- Use `PreToolUse` hooks to block dangerous Bash commands
- Principle of least privilege — grant only needed tools
- Use HTTPS for remote server transport
- Audit log all sensitive operations
- Rate-limit tool calls in production
- Never commit `.env` files (API keys)

Common Gotchas

- All MCP ops are **async** — don't forget `await`
- Use **absolute paths** — relative paths break when server runs from a different directory
- Resource URIs must be valid: `file:///path` not `/path`
- "Method not found" → server doesn't implement that capability
- "Invalid params" → check your JSON schema / type hints
- MCP tools are **not** auto-approved by permission modes — allow-list them: `allowed_tools=["mcp__x__*"]`
- SDK skills need `setting_sources=["user","project"]` or they load nothing
- Sessions are cwd-sensitive: `resume` silently starts fresh if the directory changed
- Serverless MCP: use `stateless_http=True, json_response=True`

7. KEY COMMANDS & TOOLS

Running Demos with UV

```
# UV handles all dependencies automatically
uv run demos/01-introduction-to-mcp/mcp_server.py

# Test any MCP server interactively
mcp dev demos/01-introduction-to-mcp/mcp_server.py
# Opens browser: tool test + message inspector

# Connect a server to Claude Code
claude mcp add research -- \
  uv run /abs/path/mcp_server.py
claude mcp add prod --transport http \
  https://your-app.vercel.app/mcp

# Check Claude Desktop logs (macOS)
~/Library/Logs/Claude/
```

Claude Desktop Config

```
// macOS: ~/Library/Application Support/Claude/
// claude_desktop_config.json
// Windows: %APPDATA%\Claude\claude_desktop_config.json
{
  "mcpServers": {
    "my-server": {
      "command": "uv",
      "args": [
        "run",
        "/absolute/path/to/server.py"
      ]
    }
  }
}

// After editing: restart Claude Desktop
// Use absolute paths — relative paths fail
```

8. ECOSYSTEM HACKS & RESOURCES

Build MCP Servers with AI

Use Claude Code to scaffold servers. Paste MCP SDK docs + requirements into the prompt. Iterate on one capability at a time.

Tip: Use Context7 MCP server for always-up-to-date SDK docs inside Claude Code.

LLM-Ready Repo Data

gitingest.com — Replace `github.com` with `gitingest.com` in any repo URL to get a single LLM-ready text dump of the whole repo.

deepwiki.com — Instant docs page for any GitHub repo.

Doc Indexing

llmstxt.org — Standard for AI-readable doc index files. Add `/llms.txt` to any site to make it LLM-navigable.

Check if a site has it: `site.com/llms.txt`

Key References	
Resource	**URL**
MCP Specification	modelcontextprotocol.io/specification
MCP Python SDK	github.com/modelcontextprotocol/python-sdk
Claude Agent SDK (Python)	github.com/anthropics/claude-agent-sdk-python
Community MCP Servers	github.com/modelcontextprotocol/servers
MCP Security Best Practices	modelcontextprotocol.io/specification/.../security_best_practices
MCP Apps Extension	modelcontextprotocol.io/extensions/apps
Official MCP Registry	registry.modelcontextprotocol.io
Agent Skills Standard	agentskills.io
Course Repository	github.com/EnkrateiaLucca/mcp-course

KEY TAKEAWAYS

- **MCP = USB-C for AI:** One protocol, every host — the server from module 01 runs unmodified in Claude Desktop, Claude Code, the Agent SDK, and Cursor
- **Primitives:** Tools (model runs), Resources (app exposes), Prompts (user invokes) — plus MCP Apps for in-chat UIs; sampling/roots/logging are being deprecated
- **Skills vs MCP:** MCP = access, skills = procedural know-how, plugins bundle both
- **Design tools around intent:** few well-named tools beat an API mirror — even with tool search on by default
- **Security first:** validate paths, PreToolUse hooks, least-privilege `allowed_tools`, treat tool descriptions as untrusted input
- **Deploy the server, not a wrapper:** `stateless_http=True` + `mcp.streamable_http_app()` runs anywhere — no FastAPI plumbing
- **UV handles deps:** Inline `# /// script` headers mean zero setup — `uv run server.py` just works; pin `mcp⩾2.3,<3`

© 2026 Automata Learning Lab | "Building AI Agents with MCP: From Agent Loop to Production Servers" | O'Reilly Live Training